Student ID: 112006234

I have done this assignment without the use of any AI agent or tool

Week 3 Report

Reusable function for standardizing data:

```r
standardize <- function(numbers) {
  (numbers - mean(numbers)) / sd(numbers) # formula to standardize data
}
```

Code 0.1. Reusable function to standardize data for Q1.

Sample Code for Q3 and 1(b):

```r
# load booking datetime data
bookings <- read.table("first_bookings_datetime_sample.txt", header=TRUE)

bookings$datetime[1:9] # first 9 booking timestamps

# hour from each booking time
hours  <- as.POSIXlt(bookings$datetime, format="%m/%d/%Y %H:%M")$hour
# minute from each booking time
mins   <- as.POSIXlt(bookings$datetime, format="%m/%d/%Y %H:%M")$min

minday <- hours*60 + mins # convert booking time to minutes of the day (0-1440)
plot(density(minday), main="Minute (of the day) of first ever booking", col="blue", lwd=2)
```

Code 0.2. Sample code for Q3 and 1(b).

Q1.(a) Create a normal distribution (mean=940, sd=190) and standardize.

```r
rdata <- rnorm(n = 2000, mean = 940, sd = 190) # larger n for clearer distribution

# standardize the data
rnorm_std <- standardize(rdata)
```

Code 1.1. Code for Q1(a).

(a) i) Expectation of the mean and standard deviation of rnorm_std

- ✓ The mean of rnorm_std should be 0 and the standard deviation should be 1.
- ✓ According to the properties of standardization, if you standardize all of the observations in your sample, the resulting variable (*rnorm_std* in our case) will have mean 0 and standard deviation 1.
- ✓ Proof:

```
mean(rnorm_std)
```

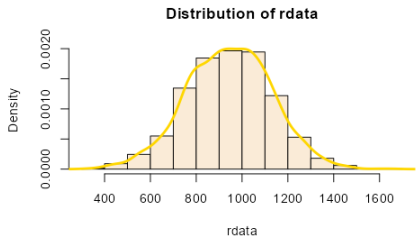
```
[1] 1.467853e-16 # approx. 0 due to floating-point error
```

```
sd(rnorm_std)
```

```
[1] 1 # standard deviation is 1
```

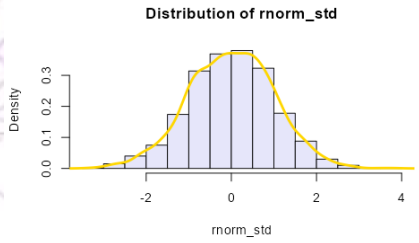
(a) ii) Shape of rnorm_std

- ✓ The distribution should still be normal (bell-shaped), the same shape as the distribution of *rdata*.
- ✓ According to the property of the normal distribution, if X (*rdata*) follows a normal distribution then the standardized variable Z (*rnorm_std*) follows the standard normal distribution.
- ✓ Proof:




```
hist(rdata, prob = TRUE, col = "antiquewhite", main = "Distribution of rdata")

lines(density(rdata), col = "gold", lwd = 3)
```




```
hist(rnorm_std, prob = TRUE, col = "lavender", main = "Distribution of rnorm_std")

lines(density(rnorm_std), col = "gold", lwd = 3)
```

(a)iii) What do we generally call distributions that are normal and standardized?

- ✓ Standard normal distribution (z-distribution)

(b) Create a standardized version of *minday*

```
# standardize minday
minday_std <- standardize(minday)
```

Code 1.2. Code for Q1(b).

(b)i) Expectation of the mean and standard deviation of *minday_std*.

- ✓ The mean of *minday_std* should be 0 and the standard deviation should be 1 for the same reason as (a)i).
- ✓ Proof:

```
mean(minday_std)
```

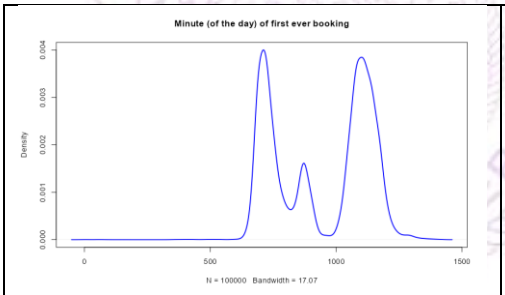
```
[1] -4.25589e-17 # approx. 0 due to floating-point error
```

```
sd(minday_std)
```

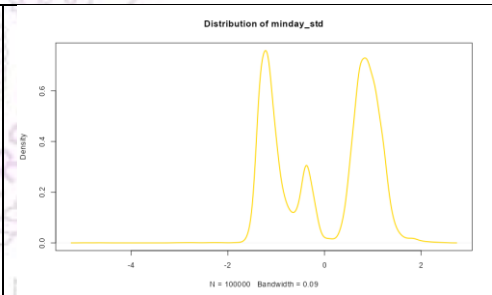
```
[1] 1 # standard deviation is 1
```

(b)ii) Distribution of *minday_std* compared to *minday*.

- ✓ The distribution of *minday_std* should have the same shape as *minday*, but it will be centered at 0 with standard deviation 1.
- ✓ This is because standardization is a linear transformation, which changes the location and scale of the data but does not change the distribution's shape.
- ✓ Proof:




```
plot(density(minday),
main="Minute (of the day) of
first ever booking",
col="blue", lwd=2)
```




```
plot(density(minday_std),
main="Distribution of
minday_std", col="gold",
lwd=2)
```

Q2. (a) Simulate 100 samples (each of size 100), from a normally distributed population of 10,000

```
plot_sample_ci(num_samples = 100, sample_size = 100,
pop_size=10000, distr_func=rnorm, mean=20, sd=3)
```

Code 2.1. Code for Q2(a).

(a)i) Expectation of samples to NOT include the population mean in its 95% CI.

- ✓ About 5 samples.
- ✓ A 95% confidence interval means that 95% of the intervals should contain the true population mean, so about 5% will NOT contain it (100×0.05=5).
- ✓ Proof:

  It is difficult to verify the theory from the plot alone because the intervals overlap and the results depend on random sampling. Therefore, I made a simulation of repeating the sampling process many times, then finding the average:

```
# repeat the experiment 200 times
miss95 <- replicate(200, {
  population <- rnorm(10000, mean=20, sd=3)
  population_mean <- mean(population) # true population mean

  # count how many of 100 samples miss the population mean
  sum(replicate(100, {
    samp <- sample(population, 100) # sample of size 100
    standard_error <- sd(samp)/sqrt(100) # compute standard error

    # 95% CI formula (z-value: 1.96)
    ci <- mean(samp) + c(-1.96,1.96)*standard_error

    # check if CI misses the true mean
    population_mean < ci[1] || population_mean > ci[2]
  }))
})

# calculate average
mean(miss95)
```

```
[1] 4.95
```

(a)ii) Expectation of samples to NOT include the population mean in its 99% CI.

- ✓ About 1 sample.
- ✓ A 99% confidence interval means 99% of intervals contain the true mean, so 1% will miss it (100×0.01=1).
- ✓ Proof:

```
# repeat the experiment 200 times
miss99 <- replicate(200, {
  population <- rnorm(10000, mean=20, sd=3)
  population_mean <- mean(population) # true population mean

  # count how many of 100 samples miss the population mean
  sum(replicate(100, {
    samp <- sample(population, 100) # sample of size 100
    standard_error <- sd(samp)/sqrt(100) # compute standard error

    # 99% CI formula (z-value: 2.58)
    ci <- mean(samp) + c(-2.58,2.58)*standard_error

    # check if CI misses the true mean
    population_mean < ci[1] || population_mean > ci[2]
  }))
})

# calculate average
mean(miss99)
```

```
[1] 1.055
```

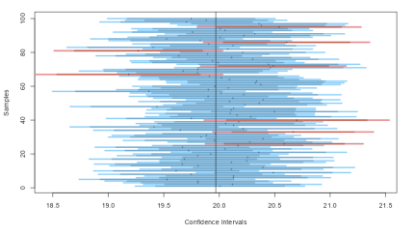
(b) Rerun the previous simulation with the same number of samples, but larger sample size (sample_size=300).

```
plot_sample_ci(num_samples = 100, sample_size = 300,
pop_size=10000, distr_func=rnorm, mean=20, sd=3)
```

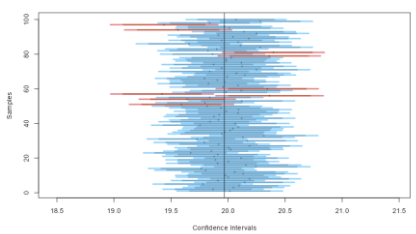
Code 2.2. Code for Q2(b).

(b)i) Do we expect their 95% and 99% CI to become wider or narrower than before?

- ✓ Narrower.
- ✓ When the sample size increases, the standard error (= sample standard deviation/sqrt(sample size)) gets smaller, so the CI becomes narrower.
- ✓ Proof:



```
plot_sample_ci(num_samples =
100, sample_size = 100,
pop_size=10000,
```



```
plot_sample_ci(num_samples =
100, sample_size = 300,
```

distr_func=rnorm, mean=20, sd=3)	pop_size=10000, distr_func=rnorm, mean=20, sd=3)

(b)ii) How many samples (out of the 100) would we expect to NOT include the population mean in its 95% CI?

- ✓ 5 out of 100.
- ✓ 95% confidence interval contains the true mean about 95% of the time, so even though the intervals become narrower, the method is still designed so that about 95 out of 100 intervals contain the true mean.
- ✓ Proof:

```
miss95_b <- replicate(200, {
  population <- rnorm(10000, mean=20, sd=3)
  population_mean <- mean(population)

  sum(replicate(100, {
    samp <- sample(population, 300) # sample of size 100
    standard_error <- sd(samp)/sqrt(300)

    ci <- mean(samp) + c(-1.96,1.96)*standard_error

    population_mean < ci[1] || population_mean > ci[2]
  }))
})

mean(miss95_b)
```

```
[1] 4.85
```

(c) If we ran the above two examples (a and b) using a uniformly distributed population, how do you expect your answers to (a) and (b) to change?

- ✓ They would not change much.
- ✓ Confidence intervals for the mean rely on the Central Limit Theorem. Even if the population is uniformly distributed instead of normal, the sampling distribution of the sample mean becomes approximately normal when the sample size is large. Therefore:
  - The expected number of samples that do not include the population mean in their 95% CI would still be about 5 out of 100, and about 1 out of 100 for the 99% CI.

- Proof:

```
miss95_c <- replicate(200, {
  population <- runif(10000) # uniform population
  population_mean <- mean(population)

  sum(replicate(100, {
    samp <- sample(population, 100)
    standard_error <- sd(samp)/sqrt(100)

    ci <- mean(samp) + c(-1.96,1.96)*standard_error

    population_mean < ci[1] || population_mean > ci[2]
  }))
})

# calculate average
mean(miss95_c)
```

```
[1] 4.97
```

```
miss99_c <- replicate(200, {
  population <- runif(10000) # uniform population
  population_mean <- mean(population)

  sum(replicate(100, {
    samp <- sample(population, 100)
    standard_error <- sd(samp)/sqrt(100)

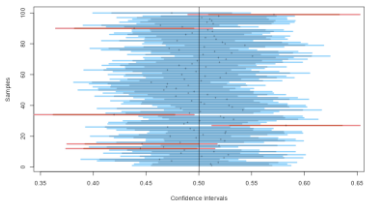
    ci <- mean(samp) + c(-2.58,2.58)*standard_error

    population_mean < ci[1] || population_mean > ci[2]
  }))
})

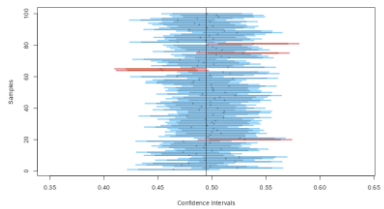
# calculate average
mean(miss99_c)
```

```
[1] 1.11
```

- Increasing the sample size would still make the confidence intervals narrower, because the standard error decreases as sample size increases.
  - Proof:



```
plot_sample_ci(num_samples = 100, sample_size = 100, pop_size=10000, distr_func=runif)
```



```
plot_sample_ci(num_samples = 100, sample_size = 300, pop_size=10000, distr_func=runif)
```

Q3. (a)i) Use traditional statistical methods to estimate the population mean of *minday*, its standard error, and the 95% confidence interval (CI) of the sampling means.

```
mean_minday <- mean(minday) # estimation of population mean
se_minday <- sd(minday) / sqrt(length(minday)) # standard error

ci_lower <- mean_minday - 1.96 * se_minday # lower limit of 95% CI
ci_upper <- mean_minday + 1.96 * se_minday # upper limit of 95% CI
```

Code 3.1. Code for Q3(a)i).

```
> mean_minday
[1] 942.4964

> se_minday
[1] 0.5997673

> c(ci_lower, ci_upper)
[1] 941.3208 943.6719
```

Output 3.1. Output for Q3(a)i).

(a)ii) Bootstrap to produce 2000 new samples from the original sample.

```
compute_sample_mean <- function(sample0){
  # bootstrap resample
  resample <- sample(sample0, length(sample0), replace=TRUE)
  mean(resample) # mean of resampled data
}

# generate 2000 bootstrap sample means
bootstrap_means <- replicate(2000, compute_sample_mean(minday))
```

Code 3.2. Code for Q3(a)ii).

(a)iii) Visualize the means of the 2000 bootstrapped samples.

```
hist(bootstrap_means,
     main="Means of the 2000 bootstrapped samples",
     col="lavender")
```

Code 3.3. Code for Q3(a)iii).

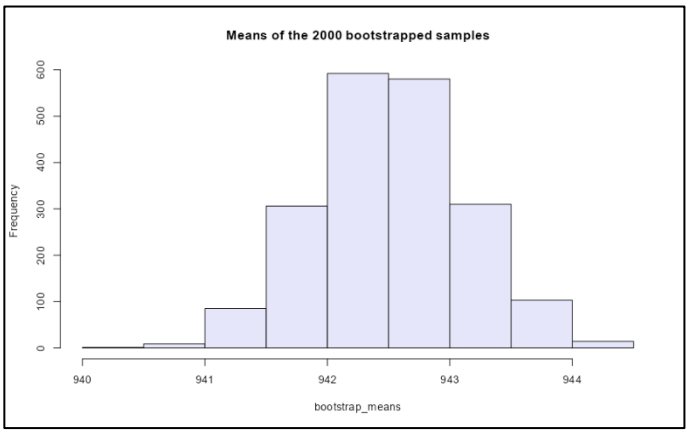


Figure 3.1. Visualization for Q3(a)iii).

Even though the data comes from a continuous variable, a histogram is used because the 2000 bootstrapped means or medians are still a set of sampled values. A histogram clearly shows how these values are distributed.

(a)iv) Estimate the 95% CI of the bootstrapped means using the quantile function.

```
quantile(bootstrap_means, c(0.025, 0.975)) # take the middle 95%
```

Code 3.4. Code for Q3(a)iv).

```
    2.5%    97.5%
941.3262 943.7035
```

Output 3.2. Output for Q3(a)iv).

(b)i) Estimate the median of minday.

```
median_minday <- median(minday)
```

Code 3.5. Code for Q3(b)i).

```
> median_minday
[1] 1040
```

Output 3.3. Output for Q3(b)i).

(b)ii) Visualize the medians of the 2000 bootstrapped samples.

```
compute_sample_median <- function(sample0){
  resample <- sample(sample0, length(sample0), replace=TRUE)
  median(resample)
}

# generate 2000 bootstrap medians
bootstrap_medians <- replicate(2000,
compute_sample_median(minday))

hist(bootstrap_medians,
     main="Medians of the 2000 bootstrapped samples",
     col="lavender")
```

Code 3.6. Code for Q3(b)ii).

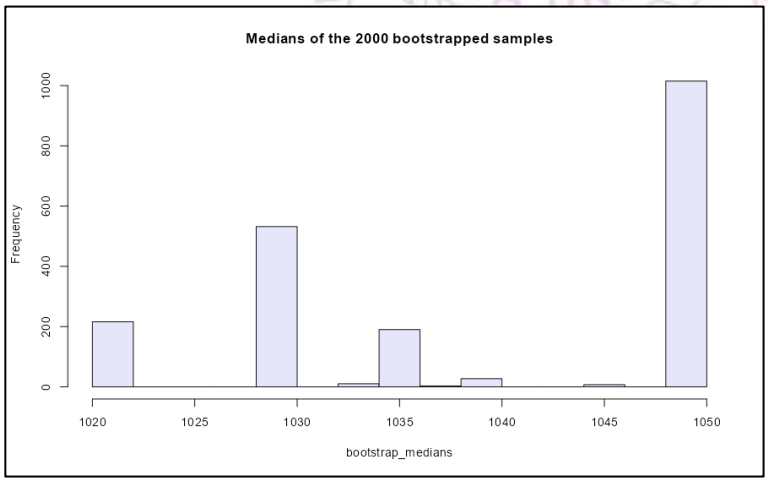


Figure 3.2. Visualization for Q3(b)ii).

The histogram also lets us see gaps in the distribution, which happens when many bootstrap samples produce the same median. A density curve smooths the data, so these gaps would not be clearly visible.

(b)iii) Estimate the 95% CI of the bootstrapped medians using the quantile function.

```
quantile(bootstrap_medians, c(0.025, 0.975))
```

Code 3.7. Code for Q3(b)iii).

```
2.5% 97.5%
 1020  1050
```

Output 3.3. Output for Q3(b)iii).